supports local storage through the *Storage* module.

```
<inherits name ='com.google.gwt.storage.client.Storage'/>
Storage storage = Storage.getLocalStorageIfSupported();
if (storage != null) {
   storage.setItem("html5", "local storage");
String value = storage.getItem("html5");
}
```

GWT provides a series of classes and events, such as *LocalStorage*, *SessionStorage*, *StorageMap* and so on, for easy access to client storage.

HTML5 visualisation via Canvas

GWT provides classes supporting the addition of 2D shapes to the application. Initialise a GWT canvas, calculate coordinates for a shape, and draw on the canvas. The assigned coordinates are scaled to their equivalent DOM size. Small coordinate shapes will give higher performance, but lag in quality. Similarly, large coordinate shapes give higher quality, but take time to initialise. GWT provides a series of classes for adding gradients, patterns and text styles, defined in the *com.google.gwt.canvas.dom.client* package. All the GWT widgets or Canvas elements added in the applications, support drag-and-drop functionality.

Embedding audio/video in the application

GWT provides classes for embedding static audio/video files, or playing them while loading the application. The audio class in the *com.google.gwt.media.client* package provides support for adding audio files. Video file functionality is provided by the *com.google.gwt.media.client.Video* class. File formats supported by these classes vary depending on the browser.

```
Audio audio = Audio.createIfSupported();
if (audio == null) {
   return;
}
audio.addSource("path/file.mp3", AudioElement.TYPE_MP3);
audio.setControls(true)

// for preloading
audio.setSrc("path/file.mp3");
audio.setPreload(MediaElement.PRELOAD_AUTO);
```

Performance and compile-time improvement

GWT not only defines the way to develop an application, but also provides guidance on improving the performance of an application. The deployed application's initial loading or start-up time increases proportionally with the size of the app. Similarly, for the development of large-scale applications, GWT compilation with the default setting will take a lot of time and processor cycles for the compilation of Java code to JavaScript. A few approaches to reduce this time are discussed below.

Code splitting

The size of an AJAX application's final output of JavaScript increases with the complexity of the application. This, in turn, increases the application's initial load time. Since the application will try to download the entire JavaScript content during initial loading, the application start-up time will increase. The code splitting methodology provides a way to define a part of the application to be downloaded initially; other parts of the application will be downloaded on demand. For instance, during the initial loading of an application, only the JavaScript required for the login page of the application is needed. Only if the user successfully logs into the application will the next page's JavaScript content be downloaded. Until then, the JavaScript content of the home page or other internal modules is not required or used.

```
//for manual loading by configuring in *.gwt.xml file
<extend-configuration-property name="compiler.splitpoint.initial.
sequence" value="com.name.of.the.class"/>

//for programmatic configuration
GWT.runAsync(new RunAsyncCallback() {
   @Override
   public void onSuccess() {
      // Load corresponding module or segment
   }
   @Override
   public void onFailure(Throwable reason) {
      // Can occur due to network failure or server
unavailability.
   }
});
```

Using the code-splitting approach, we can define the modules that need to be downloaded initially by default, and all other modules can be configured to download on demand, during the first visit. This reduces start-up time, and optimises the performance of the application.

To split the code, insert a *GWT.runAsync()* block into the required page/view initialisation of the application. This *runAsync* implements a *RunAsyncCallback* interface, which annotates the GWT compiler to create a separate module for the part covered in the block. Hence, the initial download will not include the content for the module defined in Async block. But if the initial load requires some part defined in another Async block, it will be downloaded automatically during initial load. Hence, care has to be taken while initialising split points to not mix different modules. We can even manually define the load sequence by defining them in the module descriptor file.

Browser-specific build and compile

By default, the GWT compiler produces JavaScript output for six specific browsers such as IE6, IE8, IE9, Opera, Safari and Firefox. Different combinations of output are generated